

Kernel-Quellcode und Lizenzierung

Quellbaum verstehen, SPDX lesen und GPL-Pflichten richtig einordnen

1.07

Herkunft und Release-Zweig zuerst bestimmen

UPSTREAM

Mainline

Referenzbaum für die laufende Kernelentwicklung und neue Funktionen.

WARTUNG

Stable / Longterm

Ausgewählte Fehlerkorrekturen und länger gepflegte Release-Zweige.

PRODUKT

Vendor / Distribution

Zusätzliche Backports, BSPs, Treiber und projektspezifische Anpassungen.

Relevant ist der tatsächlich ausgelieferte Quellstand – nicht nur die Upstream-Version.

Quellcode beziehen und Stand reproduzierbar sichern

UPSTREAM-QUELLE

```
$ git clone \
  https://git.kernel.org/pub/scm/linux/\
  kernel/git/torvalds/linux.git
$ cd linux
$ git rev-parse HEAD
$ git describe --tags --always
$ git status --short
```

01 Commit dokumentieren

Der Commit-Hash identifiziert den exakten überprüften Quellstand.

02 Arbeitsbaum prüfen

Lokale Änderungen und Herstellerpatches müssen nachvollziehbar bleiben.

Für ausgelieferte Produkte den tatsächlichen Vendor- oder Distributionsbaum verwenden.

Der Quellbaum organisiert Kernel-Subsysteme

arch/

Architekturabhängiger Code

kernel/

Prozesse, Scheduler, Kernlogik

mm/

Virtueller Speicher und Paging

fs/

Dateisysteme und VFS

net/

Netzwerk-Stack und Protokolle

drivers/

Geräte- und Hardwaretreiber

include/

Header und Schnittstellen

Documentation/

Technische Kernel-Dokumentation

LICENSES/

Lizenztexte und Ausnahmen

Dateien, Historie und Lizenzhinweise untersuchen

ANALYSE

```
$ git log -5 -- fs/read_write.c
$ git blame fs/read_write.c
$ git grep 'SYSCALL_DEFINE' -- fs/
$ git grep 'SPDX-License-Identifier' \
  -- drivers/usb/
$ sed -n '1,18p' COPYING
```

01 Historie

log und blame ordnen Änderungen, Commit und Herkunft ein.

02 Lizenzkontext

COPYING und SPDX-Zeilen müssen im konkreten Quellstand zusammen gelesen werden.

Ein einzelner Lizenzstring ersetzt weder den vollständigen Text noch die Dateiprüfung.

Der Linux-Kernel steht unter GPL Version 2 only

KERNEL-QUELLBAUM

GPL-2.0

Projektinterner Identifier nach den offiziellen Kernel-Lizenzregeln.

AKTUELLE SPDX-LISTE

GPL-2.0-only

Heutiger SPDX-Kurzname für die Lizenz ohne „or later“-Option.

BEDEUTUNG

Nur Version 2

Keine automatische Wahl von GPLv3 oder einer späteren Version.

GPL-2.0+ bedeutet im Kernel dagegen: Version 2 oder eine spätere Version.

SPDX kennzeichnet die Lizenz jeder Quelldatei

DATEIANFANG

```
// SPDX-License-Identifier: GPL-2.0

// SPDX-License-Identifier: GPL-2.0+

// SPDX-License-Identifier: GPL-2.0 OR MIT

/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note
*/
```

01 Maschinenlesbar

Der Identifier steht möglichst in der ersten kommentierbaren Zeile.

02 Dateityp beachten

C-Dateien, Header und Skripte verwenden jeweils passende Kommentarformen.

OR, AND und WITH verändern die Lizenzlogik

WAHLRECHT

OR

GPL-2.0 OR MIT

Eine angebotene Lizenz wird gewählt; die Optionen nicht vermischen.

KUMULATION

AND

GPL-1.0+ AND LGPL-2.1+

Die Bedingungen beider bezeichneten Lizenzen sind einzuhalten.

ZUSATZREGEL

WITH

GPL-2.0 WITH
Linux-syscall-note

Die bezeichnete Ausnahme ergänzt die zugehörige Lizenz.

Operatoren sind Teil der Lizenzangabe – nicht bloß alternative Schreibweisen.

COPYING und LICENSES liefern den Volltext

LIZENZSTRUKTUR

```
COPYING
LICENSES/
├── preferred/
│   ├── GPL-2.0
│   └── MIT
├── dual/
├── deprecated/
├── exceptions/
│   └── Linux-syscall-note
```

01 COPYING

Gesamtlizenz des Kernelprojekts und Verweis auf die System-Call-Ausnahme.

02 LICENSES/

Volltexte, Metadaten und Hinweise für bevorzugte, duale und auslaufende Lizenzen.

Ein SPDX-Identifizierer verweist auf Lizenzinhalt – er ersetzt den Volltext nicht.

System Calls bilden eine klare User-Space-Grenze

USER SPACE

Unabhängige Anwendung

Normale Systemaufrufe machen ein eigenständiges Anwenderprogramm nicht allein deshalb zu Kernelcode.

KERNEL SPACE

GPLv2-Kernel

Die Ausnahme beschreibt die reguläre Nutzung der Kernel-Schnittstelle; sie lizenziert nicht beliebigen kopierten Kernelcode um.

UAPI-Header: GPL-2.0 WITH Linux-syscall-note

Die System-Call-Ausnahme ist keine pauschale Ausnahme für Kernelmodule.

Kernelmodule benötigen eine eigene Grenzprüfung

AUSFÜHRUNG

Kernel Space

Ladbare Module laufen innerhalb des Kernelkontexts und verwenden interne Schnittstellen.

BEWERTUNG

Einzelfall

Kopierter Kernelcode, Art der Integration und verwendete Symbole müssen konkret geprüft werden.

RISIKOSTEuerung

Legal Review

Out-of-tree ist kein automatischer Freibrief und kein automatischer Beweis für eine bestimmte Einordnung.

Technische Ladebarkeit entscheidet nicht abschließend über urheberrechtliche Pflichten.

MODULE_LICENSE ist ein technisches Loader-Signal

MODUL UND SYMBOL

```
// SPDX-License-Identifier: GPL-2.0
MODULE_LICENSE("GPL");

EXPORT_SYMBOL_GPL(kernel_service);

$ cat /proc/sys/kernel/tainted
$ modinfo -F license mymodule.ko
```

01 Keine Lizenzurkunde

MODULE_LICENSE ersetzt weder SPDX noch die tatsächliche Lizenz der Quelldatei.

02 GPL-only und Taint

Proprietäre Module erhalten P-Taint und können GPL-only-Symbole nicht verlinken.

„GPL“ im Modul-Tag unterscheidet nicht zwischen GPLv2-only und GPLv2-or-later.

Bei Weitergabe greifen konkrete GPLv2-Pflichten

01 Lizenz- und Copyright-Hinweise

Lizenztext, relevante Copyright-Hinweise und Änderungen nachvollziehbar weitergeben.

02 Korrespondierenden Quellcode bereitstellen

Passender Quellstand einschließlich relevanter Schnittstellen und Build-/Installationsskripte.

03 Zulässigen Bereitstellungsweg wählen

GPLv2 § 3 nennt Quellcode-Mitlieferung oder ein regelkonformes schriftliches Angebot.

Ein bloßer Link auf irgendeinen Upstream-Tree ersetzt keinen passenden Produktquellstand.

Interne Nutzung ist nicht dasselbe wie Weitergabe

REGELMÄSSIG KEINE WEITERGABE

Interner Betrieb

Das Ausführen eines Kernels und rein interne Nutzung lösen nicht allein dadurch die Pflichten der Binärweitergabe aus.

COMPLIANCE VOR AUSLIEFERUNG

Abgabe an Dritte

Kernel-Binärpakete, Geräteimages oder ausgelieferte Produkte sind auf GPLv2-konforme Weitergabe zu prüfen.

Wer Empfänger ist und ob tatsächlich weitergegeben wird, hängt vom konkreten Fall ab.

Praxis-Check: Quellcode und Lizenzierung sichern

01 Herkunft festhalten

Vendor-Tree, Commit, Version, Konfiguration und sämtliche Produktpatches dokumentieren.

02 Dateilizenzen prüfen

COPYING, LICENSES, SPDX-Ausdrücke, UAPI-Header und externe Komponenten abgleichen.

03 Auslieferung vorbereiten

Korrespondierenden Quellcode, Lizenzhinweise und den Bereitstellungsweg kontrollieren.

04 Grenzfälle eskalieren

Kernelmodule, proprietäre Komponenten und konkrete Lieferketten rechtlich prüfen lassen.

Schulungsorientierung – keine Rechtsberatung; konkrete Entscheidungen mit Compliance abstimmen.